

The `hooks` parameter defines commands that execute automatically at specific lifecycle events, enabling automation and context gathering for enhanced agent behavior. It accepts a JSON object with trigger names as keys (`"agentSpawn"` for agent initialization, `"userPromptSubmit"` for each user message) and arrays of hook objects as values, where each hook contains a required `command` string that gets executed as a shell command.

The output from these commands is automatically added to the agent's context, allowing agents to gather real-time information like git status, project file listings, or environment details that inform their responses, while hooks execute in the background without requiring user permission and can be used for logging, monitoring, setup tasks, or dynamic context updates throughout the conversation.

## Objective

Learn how to use `hooks` parameter to automate tasks when agents start or receive prompts.

## Instructions

### Step 8.1: Create Automation Scripts

#### 1. Create logging script

```
1 cat > scripts/log-interaction.sh << 'EOF'
2 #!/bin/bash
3 echo "$(date): Agent interaction logged" >> .kiro/agent-activity.log
4 echo "User: $1" >> .kiro/agent-activity.log
5 echo "----" >> .kiro/agent-activity.log
6 EOF
```

```
1 chmod +x scripts/log-interaction.sh
```

#### 2. Create startup script

```
1 cat > scripts/agent-startup.sh << 'EOF'
2 #!/bin/bash
3 echo "$(date): Advanced agent started" >> .kiro/agent-activity.log
4 echo "Project: $(basename $(pwd))" >> .kiro/agent-activity.log
5 echo "Working directory: $(pwd)" >> .kiro/agent-activity.log
6 echo "Agent files: $(find .kiro/agents -name '*.json' 2>/dev/null | wc -l) agents configured" >> .kiro/agent-activity.log
7 echo "----" >> .kiro/agent-activity.log
8 EOF
```

```
1 chmod +x scripts/agent-startup.sh
```

### Step 2: Create Agent with Lifecycle Hooks

```
1 cat > .kiro/agents/automated-dev.json << 'EOF'
2 {
3   "name": "automated-dev",
4   "description": "Development agent with automation hooks",
5   "prompt": "You are an advanced development assistant with automated logging and startup procedures.",
6   "tools": ["read", "write", "shell"],
7   "allowedTools": ["read"],
8   "resources": [
9     "file://README.md",
10    "file://.kiro/agent-activity.log"
11  ],
12  "hooks": {
13    "agentSpawn": [
14      {
15        "command": "./scripts/agent-startup.sh"
16      }
17    ],
18    "userPromptSubmit": [
19      {
20        "command": "./scripts/log-interaction.sh \"$USER_PROMPT\""
21      }
22    ]
23  }
24 }
25 EOF
```

### Step 3: Test Lifecycle Hooks

#### 1. Start custom agent

```
1 kiro-cli chat --agent automated-dev
```

#### 2. Commands to try:

```
"Hello, I'm testing the automated agent"
"What project files can you see?"
"Check the agent activity log"
```

### Step 4: Verify Hook Execution

#### 1. Check the log file

```
1 cat .kiro/agent-activity.log
```

### Step 5: Create Agent with Multiple Hooks

#### 1. Create project status script

```
1 cat > scripts/project-status.sh << 'EOF'
2 #!/bin/bash
3 echo "$(date): Project status check" >> .kiro/agent-activity.log
4 echo "Files: $(find . -name '*.js' | wc -l) JavaScript files" >> .kiro/agent-activity.log
5 echo "Docs: $(find docs -name '*.md' 2>/dev/null | wc -l) documentation files" >> .kiro/agent-activity.log
6 echo "----" >> .kiro/agent-activity.log
7 EOF
```

```
1 chmod +x scripts/project-status.sh
```

#### 2. Create multiple hooks agent

```
1 cat > .kiro/agents/multi-hook-agent.json << 'EOF'
2 {
3   "name": "multi-hook-agent",
4   "description": "Agent with multiple startup hooks",
5   "prompt": "You are a project monitoring agent with comprehensive startup procedures.",
6   "tools": ["read"],
7   "allowedTools": ["read"],
8   "hooks": {
9     "agentSpawn": [
10      {
11        "command": "./scripts/agent-startup.sh"
12      },
13      {
14        "command": "./scripts/project-status.sh"
15      }
16    ]
17  }
18 }
19 EOF
```

### Step 6: Test Multiple Hooks

#### 1. Start the agent

```
1 kiro-cli chat --agent multi-hook-agent
```

#### 2. Check the log to see both hooks executed

```
1 cat .kiro/agent-activity.log
```

## Verification

- ✔ Startup hooks execute when agent launches
- ✔ User prompt hooks execute on each interaction
- ✔ Multiple hooks can be chained together
- ✔ Hook execution is logged correctly
- ✔ Scripts have proper permissions and execute successfully

## Key Learnings

- `hooks` parameter enables automation at specific lifecycle events
- `agentSpawn` hooks run when the agent starts
- `userPromptSubmit` hooks run on each user interaction
- Multiple hooks can be executed in sequence
- Hooks enable logging, monitoring, and setup automation